

ferry

Build with Hashi

Bitcoin as a Sui asset. Your BTC stays on Bitcoin and hBTC moves on Sui. Today: the Hashi SDK, an app your users can trust for the hour a deposit takes, and a Move object with its own Bitcoin address.

day3-ferry.vercel.app

[KEYS, DOLLARS AND BITCOIN · DAY 3 · SUIHUB LAGOS]

RURU (@MINTING_RURU)

[THE ONE IDEA]

Lock on Bitcoin, mint on Sui, burn to release.

Hashi does all three.
You build what the user touches.

Your BTC goes to a Taproot address that Sui's validator committee and an independent guardian control together. After 6 confirmations the committee approves it on Sui, and after a 10 minute safety delay hBTC is minted, one for one. Burn hBTC and the committee sends BTC back. On 11 November Hashi held 14.42974424 BTC against 14.42974424 hBTC.

[THE PROTOCOL]

What Hashi is made of

From Hashi's docs and Move source, with the live numbers from mainnet on 11 November.

01 Committee

105 Sui validators sign Bitcoin transactions with threshold MPC. No single key holds the BTC.

02 Guardian

A second key in every deposit address, and a rate limit out: a 50 BTC bucket, about 50 a day.

03 Your address

One Taproot address per Sui address, from the MPC key, the guardian key and your Sui address.

04 hBTC

An 8-decimal Sui coin, minted with send_funds into your address balance. One per BTC held.

05 Delays

6 Bitcoin confirmations, a 10 minute window to pause a bad approval, a 1 hour cancel cooldown.

06 Governance

No HASHI token. Committee members vote with stake on pauses, minimums and upgrades.

[READ FROM CHAIN, 11 NOVEMBER]

Hashi on mainnet and testnet

view.all(), view.withdrawalFees() and Sui GraphQL. Testnet pairs with Bitcoin Signet.

	Mainnet	Testnet
Bitcoin side	Bitcoin mainnet	Bitcoin Signet
Hashi package	0xd30f…9d06	0x5e0b…8485
Minimum in and out	30,000 sats	1 BTC (100,000,000 sats)
Confirmations, then delay	6, then 10 minutes	6, then 10 minutes
Miner fee budget per withdrawal	29,454 sats	99,999,454 sats
BTC held and hBTC minted	14.42974424 and 14.42974424	read the same way
SDK 0.6.29 defaults	none: pass the ids	built in

[@MYSTEN/HASHI 0.6.29]

The stack in one screen

The SDK extends any Sui client. tx builds unsigned PTBs, call gives Move calls, view reads.

TYPESCRIPT

```typescript
const client = new SuiGrpcClient({ network: 'mainnet', baseUrl }).$extend(
  hashi({ hashiObjectId, packageId, bitcoinNetwork: 'mainnet' }),  // mainnet: pass the ids
);

client.hashi.generateDepositAddress({ suiAddress })    // your Bitcoin address
client.hashi.view.all()                                // minimums, confirmations, delays, paused
client.hashi.view.findUsedUtxos([{ txid, vout }])      // already recorded?
client.hashi.view.depositStatus(digest)                // pending, confirmed, expired
client.hashi.view.transactionHistory(address)          // deposits and withdrawals
client.hashi.tx.deposit({ txid, utxos, recipient })    // unsigned PTB: the wallet signs
client.hashi.tx.requestWithdrawal({ amount, bitcoinAddress })
client.hashi.call.cancelWithdrawal({ requestId })      // a Move call for your own PTB
client.hashi.guardian.canWithdraw(sats)                // the rate limiter
```

[ONE ADDRESS PER SUI ADDRESS]

Your Bitcoin address

generateDepositAddress({ suiAddress }): the same address every time, and nobody's keys but Hashi's can spend it.

How it is built

- Taproot, internal key unspendable (NUMS)
- Leaf 1: guardian key and MPC child key, 2 of 2
- Leaf 2: MPC key alone, after a delay (recovery)
- Child key = derive(MPC master key, your Sui address)

What it means for your app

- Same address for the same Sui address, always
- The user's own keys cannot spend from it
- Below 30,000 sats a payment is stuck: say so
- Checked: 7 of 7 held deposits match the SDK

[DEPOSIT]

BTC in, hBTC out

On 11 November, mainnet deposits took 40 minutes to 2 and a half hours from record to mint.

01

Send

From any Bitcoin wallet to your deposit address. At least 30,000 sats per output.

02

Record

deposit::deposit on Sui with the txid and vout. Anyone can send it; hBTC goes to you.

03

Confirm

6 Bitcoin blocks, about an hour. The committee watches Bitcoin, not you.

04

Approve

The committee signs over the request and records the approval time on Sui.

05

Mint

After the 10 minute delay, confirm_deposit sends hBTC to your address balance.

Record it from the browser

Abridged from Ferry. Find the payment, build the PTB, simulate, then the user's wallet signs.

TYPESCRIPT

```typescript
// 1. Hashi does not scan Bitcoin: find the outputs that paid the address
const txs = await fetch(`${esplora}/address/${address}/txs`).then((r) => r.json());
const outs = txs[0].vout
  .map((o, vout) => ({ vout, amountSats: BigInt(o.value), to: o.scriptpubkey_address }))
  .filter((o) => o.to === address);

// 2. one PTB per Bitcoin transaction (txid as explorers show it)
const tx = client.hashi.tx.deposit({ txid: `0x${txs[0].txid}`, utxos: outs, recipient: me });

// 3. simulate the exact bytes, then the wallet signs them
tx.setSender(me);
const bytes = await tx.build({ client });
await client.core.simulateTransaction({ transaction: bytes, include: { effects: true } });
await wallet.features['sui:signTransaction'].signTransaction({ transaction: Transaction.from(bytes), account, chain });
```

[WITHDRAW]

The way back to Bitcoin

tx.balance<hBTC>(amount), then withdraw::request_withdrawal. The status comes from view.withdrawalStatus(digest).

Status	What happened	Cancel?
Requested	Your hBTC is locked in the request	After 1 hour
Approved	The committee signed off on the request	After 1 hour
Processing	In a Bitcoin batch; the hBTC is burned	No
Signed	MPC and guardian signatures recorded on Sui	No
Confirmed	The BTC landed at your Bitcoin address	No

At the minimum: 30,000 sats out, as little as 546 in. The fee budget is a ceiling; the real fee is usually far lower.

[THE TRACKER]

An hour of waiting is a state, not an error

Every stage from view.depositStatus(digest) and Esplora. Anything ambiguous shows as pending.

01

Seen

In the mempool. Recordable now: the committee waits for blocks itself.

02

Confirming

n of 6 from Esplora. About an hour on Bitcoin.

03

Approved

The committee has signed. The SDK reports when.

04

Delay

Count down to the mint time the SDK reports. A pause can hold it here.

05

Minted

Status confirmed. hBTC is in the address balance.

[MOVE/JAR/SOURCES/JAR.MOVE]

A contract with its own Bitcoin address

An object's ID is a Sui address, so Hashi can derive a Bitcoin address for it and mint hBTC to it.

MOVE

```
public struct Jar<phantom T> has key { id: UID, unlock_ms: u64, taken: u64 }

// Hashi mints with balance::send_funds(btc, jar id): the hBTC lands in the jar itself
public fun take<T>(self: &mut Jar<T>, cap: &JarCap<T>, amount: u64, clock: &Clock): Balance<T> {
    assert!(cap.jar_id == self.id.to_inner(), EWrongCap);
    assert!(clock.timestamp_ms() ≥ self.unlock_ms, ELocked);
    balance::redeem_funds(balance::withdraw_funds_from_object<T>(&mut self.id, amount))
}

public fun held<T>(self: &Jar<T>, root: &AccumulatorRoot): u64 {
    balance::settled_funds_value<T>(root, self.id.to_address())
}
// TS: client.hashi.generateDepositAddress({ suiAddress: jarId })  →  bc1p890h9...45ne7h
```

[VERIFIED]

Proof: real transactions and checks

Mainnet reads and simulations move no money. The jar ran on testnet with SUI.

What	Where	Transaction or check
A 0.02 BTC deposit, recorded to minted	Sui mainnet	`57SeRZ7xKmmRB6Tm1AgNV3Ngkb6b1VRnvzvfHbvYZP25`
A 0.0003 BTC withdrawal, BTC confirmed	Sui mainnet	`GUHmUvtZPvpEFTvavzM6HsCGab2Fid847UZAjLtTefa5`
BTC held equals hBTC supply	Sui + Bitcoin	`1,442,974,424 sats each`
Held deposits paid the derived address	Bitcoin mainnet	`7 of 7`
Ferry's record and withdraw bytes	Sui mainnet	`simulate to Requested events`
Jar funded with send_funds	Sui testnet	`FQLAhzyrsNW62Q27fKdvJJtNhauyDkB326jA4w5eXp6B`
Jar taken after unlock	Sui testnet	`3StPb2xWLz4FiStur6z5dq3fvnUwwAsFgB1R8GQe2hM8`
Our own deposit through Ferry	Sui mainnet	`pending`

[GOTCHAS]

Six things that will bite you

All of them from building Ferry against mainnet.

01 Testnet-first SDK

Pin 0.6.29 and pass the mainnet package and object ids yourself.

02 No scanning

Hashi waits for someone to record the payment on Sui. Find it for the user.

03 Reversed txids

Pass the txid explorers show. Hashi stores the bytes reversed.

04 The minimum

Under 30,000 sats a payment cannot be recorded or spent. Say so first.

05 Requests expire

Not confirmed within a day: nothing minted, BTC still there. Record again.

06 Fee is a ceiling

At the minimum the worst case leaves 546 sats. Show it before signing.

Take home: Bitcoin stays on Bitcoin. Your app finds the payment, records it, and tells the truth for the hour it takes.